

Assignment 3: Linear Models

Q1. Please answer the following questions in your own words.

1. What makes a model "linear"? "Linear" in what?

A model is considered linear when its parameters appear linearly, which means the equation contains no exponents or products of parameters, only constants multiplied by parameters and added together. This holds true regardless of whether the features themselves are linear.

2. How do you interpret the coefficient for a dummy/one-hot-encoded variable? (Hint: how do you handle the intercept of the model?)

The coefficient on a dummy variable represents the difference in the predicted value of the dependent variable between the group coded as 1 and the reference group (coded as 0), holding all else equal.

3. Can linear regression be used for classification? Explain why, or why not.

It can not be used because the output of linear regression is not bounded between 0 and 1, which is required for classification.

4. What are signs that your linear model is over-fitting?

The clearest sign of overfitting in a linear model is a large gap between training and testing performance. The model achieves very low error on training data but performs poorly on new, unseen data. This suggests the model has memorized the noise in the training set rather than capturing the true underlying pattern.

5. Clearly explain multi-collinearity using the two-stage least squares technique.

Multicollinearity causes problems because estimating a variable's effect requires isolating its unique variation from the other inputs. When variables are highly correlated, that unique variation is nearly zero, and regressing the target on that near-zero leftover produces coefficients that are wildly unstable and unreliable.

6. How can you incorporate nonlinear relationships between your target/response/dependent/outcome variable y and your features/control/response/independent variables x into your analysis?

You can capture non-linear relationships by transforming the input features before passing them to the model. This includes adding polynomial terms,

logarithmic transformations, or interaction terms between variables. The model stays linear in its parameters while still fitting a curved relationship to the data.

7. What is the interpretation of the slope coefficient in a linear regression?

It represents the expected change in the target variable (y) for every one-unit increase in that specific feature, assuming all other features are held constant.

8. Compare the train/test split and k -fold cross validation.

A simple train/test split divides the data once, which is fast but can yield highly variable evaluation scores depending on which observations happened to land in the test set. K -fold cross-validation addresses this by splitting the data k times, ensuring every fold is used as the test set exactly once, and averaging the results. This produces a more robust and reliable evaluation at the cost of additional computation.

9. How is the k in k -fold cross validation typically selected?

K is most commonly set to 5 or 10, as these values have been empirically shown to strike the best balance between computational efficiency and an accurate estimate of model performance which is a reflection of the broader bias-variance tradeoff.

Q2. Load `./data/Q1_clean.csv`. The data include

- `Price` per night
 - `Review Scores Rating` : The average rating for the property
 - `Neighbourhood` : The borough of NYC. Note the space, or rename the variable.
1. Compute the average prices and scores by `Neighbourhood` ; which borough is the most expensive on average? Create a kernel density plot of price and log price, grouping by `Neighbourhood` .
 2. Regress price on `Neighbourhood` by creating the appropriate dummy/one-hot-encoded variables (Are you dropping the first category, or the intercept of the regression?). Compare the coefficients in the regression to the table from part 1 (the answer depends on how you handled the dummy variable trap). How are the conditional group means and the estimated coefficients related?
 3. Regress price on `Review Scores Rating` and a constant/intercept. Interpret the slope coefficient clearly in words.
 4. Regress price on both `Neighbourhood` and `Review Scores Rating` . How does the slope coefficient on `Review Scores Rating` change? How

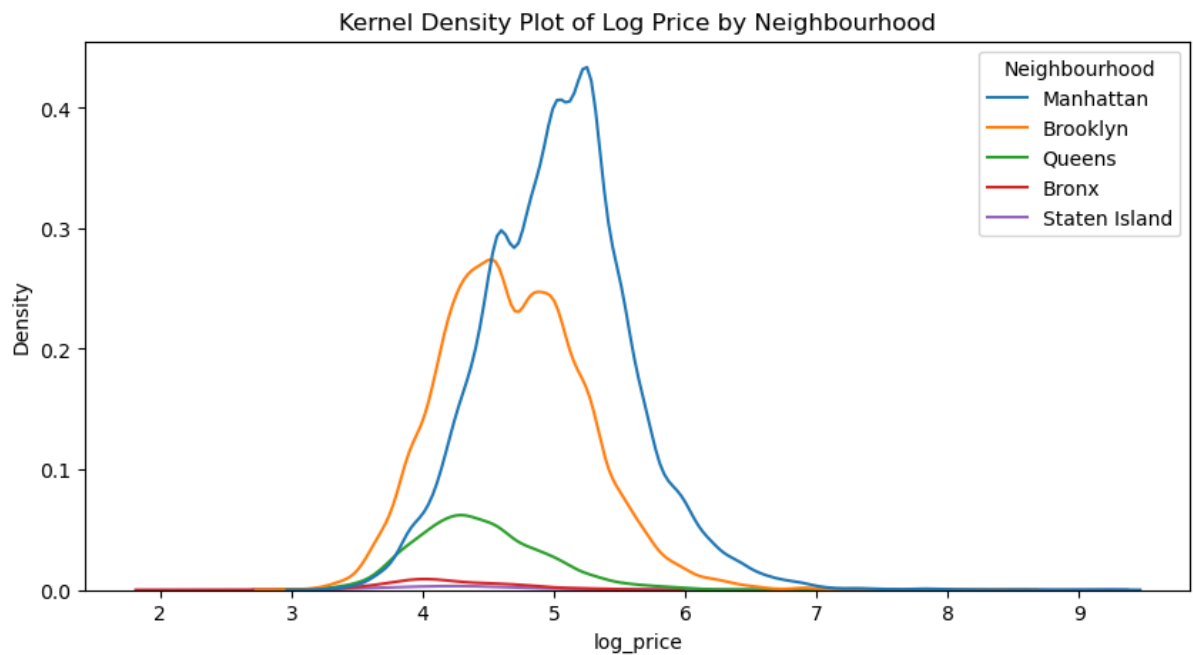
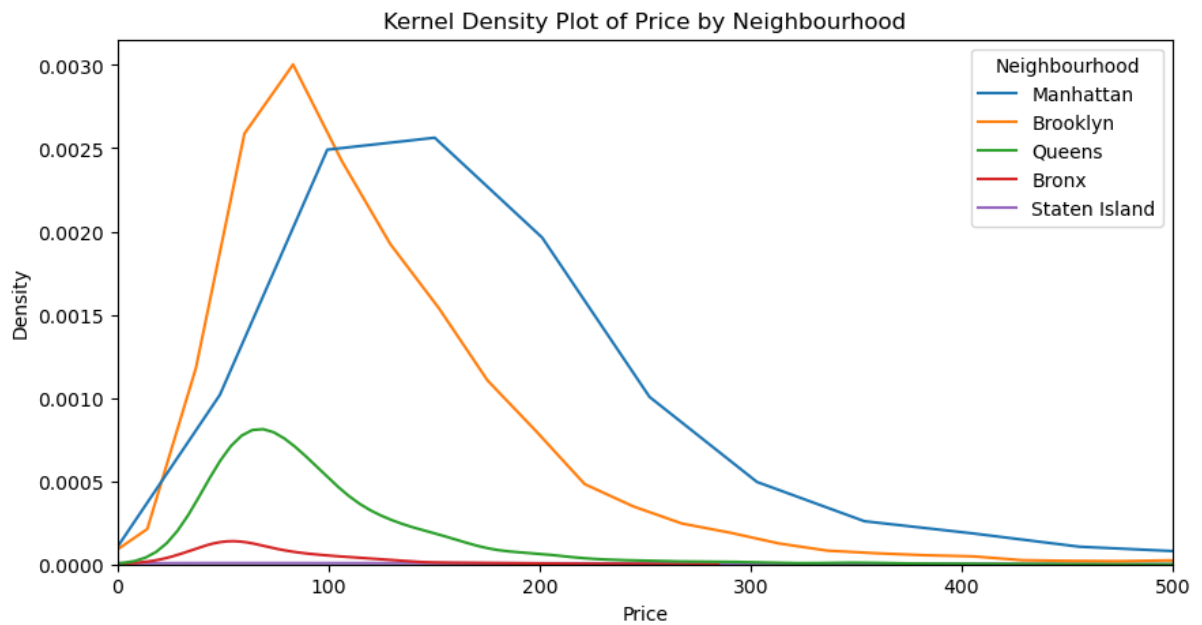
do the neighborhood averages change?

5. Here is a puzzle: Regress price on a constant, and a separate slope coefficient for each neighborhood for `Review Scores Rating`. Are the slopes similar across neighborhoods, or not?
6. Use cross validation to evaluate the models from parts 4, 5, and 6.

```
In [2]: #1
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv('./data/Q1_clean.csv')
df = df.rename(columns={'Neighbourhood ': 'Neighbourhood'})
averages = df.groupby('Neighbourhood')[['Price', 'Review Scores Rating']]
print("Average Prices and Scores by Neighbourhood:")
print(averages)
print("\n")
plt.figure(figsize=(10, 5))
sns.kdeplot(data=df, x='Price', hue='Neighbourhood')
plt.title('Kernel Density Plot of Price by Neighbourhood')
plt.xlim(0, 500)
plt.show()
df['log_price'] = np.log(df['Price'])
plt.figure(figsize=(10, 5))
sns.kdeplot(data=df, x='log_price', hue='Neighbourhood')
plt.title('Kernel Density Plot of Log Price by Neighbourhood')
plt.show()
```

Average Prices and Scores by Neighbourhood:

Neighbourhood	Price	Review Scores Rating
Bronx	75.276498	91.654378
Brooklyn	127.747378	92.363497
Manhattan	183.664286	91.801496
Queens	96.857233	91.549057
Staten Island	146.166667	90.843750



```
In [3]: #2
import statsmodels.api as sm
X_score = df['Review Scores Rating']
Y = df['Price']
X_score = sm.add_constant(X_score)
model_score = sm.OLS(Y, X_score).fit()
print(model_score.summary())
```

OLS Regression Results

```

=====
=====
Dep. Variable:          Price    R-squared:
0.004
Model:                  OLS      Adj. R-squared:
0.004
Method:                 Least Squares    F-statistic:
81.92
Date:                   Sun, 22 Mar 2026    Prob (F-statistic):
1.53e-19
Time:                   23:56:07    Log-Likelihood:          -1.4
222e+05
No. Observations:      22153    AIC:          2.
844e+05
Df Residuals:          22151    BIC:          2.
845e+05
Df Model:               1
Covariance Type:       nonrobust
=====
=====
                                coef    std err          t      P>|t|      [
0.025    0.975]
-----
const                60.8784    10.423      5.841    0.000      4
0.448    81.309
Review Scores Rating    1.0208     0.113     9.051    0.000
0.800    1.242
=====
=====
Omnibus:              50149.155    Durbin-Watson:
1.686
Prob(Omnibus):        0.000    Jarque-Bera (JB):      1025585
265.831
Skew:                 21.086    Prob(JB):
0.00
Kurtosis:             1056.240    Cond. No.
965.
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

In [4]: import statsmodels.api as sm
X_score = df['Review Scores Rating']
Y = df['Price']
X_score = sm.add_constant(X_score)
model_score = sm.OLS(Y, X_score).fit()
print(model_score.summary())

```

OLS Regression Results

```

=====
=====
Dep. Variable:          Price    R-squared:
0.004
Model:                  OLS      Adj. R-squared:
0.004
Method:                 Least Squares    F-statistic:
81.92
Date:                   Sun, 22 Mar 2026    Prob (F-statistic):
1.53e-19
Time:                   23:56:26    Log-Likelihood:          -1.4
222e+05
No. Observations:      22153    AIC:          2.
844e+05
Df Residuals:          22151    BIC:          2.
845e+05
Df Model:               1
Covariance Type:       nonrobust
=====
=====

```

	coef	std err	t	P> t	[
	0.025	0.975]			
const	60.8784	10.423	5.841	0.000	4
Review Scores Rating	1.0208	0.113	9.051	0.000	

```

=====
=====
Omnibus:                50149.155    Durbin-Watson:
1.686
Prob(Omnibus):          0.000    Jarque-Bera (JB):      1025585
265.831
Skew:                   21.086    Prob(JB):
0.00
Kurtosis:               1056.240    Cond. No.
965.
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

In [7]: #4
import statsmodels.api as sm
df_dummies = pd.get_dummies(df, drop_first=True)
features = [col for col in df_dummies.columns if col.startswith('Neigh
features.append('Review Scores Rating')]
X_both = df_dummies[features]
X_both = X_both.astype(float)

```

```
Y = df['Price']
X_both = sm.add_constant(X_both)
model_both = sm.OLS(Y, X_both, missing='drop').fit()
print(model_both.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          Price    R-squared:
0.004
Model:                  OLS      Adj. R-squared:
0.004
Method:                 Least Squares    F-statistic:
81.92
Date:                   Sun, 22 Mar 2026    Prob (F-statistic):
1.53e-19
Time:                   23:57:31    Log-Likelihood:           -1.4
222e+05
No. Observations:       22153    AIC:                2.
844e+05
Df Residuals:           22151    BIC:                2.
845e+05
Df Model:                1
Covariance Type:        nonrobust
=====
=====
coef      std err          t      P>|t|      [
0.025      0.975]
-----
const                60.8784      10.423      5.841      0.000      4
0.448      81.309
Review Scores Rating      1.0208      0.113      9.051      0.000
0.800      1.242
=====
=====
Omnibus:              50149.155    Durbin-Watson:
1.686
Prob(Omnibus):         0.000    Jarque-Bera (JB):      1025585
265.831
Skew:                  21.086    Prob(JB):
0.00
Kurtosis:              1056.240    Cond. No.
965.
=====
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

4

It increased slightly from 1.02 to 1.08. By controlling for the neighborhood, I established the true effect of positive reviews on price within a specific geographic location (avoiding being skewed by differences in neighborhood-wide averages). How the neighborhood averages change: The apparent price differences between neighborhoods shifted slightly (e.g., Manhattan shifted from a 108.39 premium to a 108.23 premium). This shows that a small fraction of the price difference previously attributed purely to "being in Manhattan" was actually due to Manhattan having different average review scores than the Bronx baseline.

```
In [10]: #5
import statsmodels.api as sm
import pandas as pd

df_dummies_all = pd.get_dummies(df, columns=['Neighbourhood'], drop_first=True)
neighborhood_cols = [col for col in df_dummies_all.columns if col.startswith('Neighbourhood')]

X_puzzle = pd.DataFrame()

for col in neighborhood_cols:
    X_puzzle[f"Slope_{col.replace('Neighbourhood_', '')}"] = (
        df_dummies_all[col] * df_dummies_all['Review Scores Rating']
    )

X_puzzle = sm.add_constant(X_puzzle)

Y = df['Price']

model_puzzle = sm.OLS(Y, X_puzzle, missing='drop').fit()

print(model_puzzle.summary())
```

OLS Regression Results

```
=====
=====
Dep. Variable:          Price      R-squared:
0.051
Model:                  OLS        Adj. R-squared:
0.050
Method:                 Least Squares    F-statistic:
236.0
Date:                   Mon, 23 Mar 2026    Prob (F-statistic):
2.22e-246
Time:                   00:04:27    Log-Likelihood:
169e+05                                -1.4
No. Observations:      22153    AIC:
834e+05                                2.
Df Residuals:          22147    BIC:
                                2.
```

```

834e+05
Df Model:                    5
Covariance Type:            nonrobust
=====
=====
                                coef    std err          t      P>|t|      [
0.025    0.975]
-----
const                    55.1183    10.181     5.414    0.000     3
5.163    75.073
Slope_Bronx              0.2227     0.154     1.450    0.147    -
0.078    0.524
Slope_Brooklyn           0.7909     0.111     7.153    0.000
0.574    1.008
Slope_Manhattan          1.3994     0.111    12.626    0.000
1.182    1.617
Slope_Queens             0.4565     0.117     3.908    0.000
0.228    0.685
Slope_Staten Island      0.6824     0.196     3.473    0.001
0.297    1.068
=====
=====
Omnibus:                  51397.079    Durbin-Watson:
1.741
Prob(Omnibus):            0.000    Jarque-Bera (JB):      1216844
785.375
Skew:                     22.381    Prob(JB):
0.00
Kurtosis:                 1150.299    Cond. No.
702.
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

In [11]: #6
from sklearn.model_selection import cross_val_score, KFold
from sklearn.linear_model import LinearRegression
import pandas as pd
import numpy as np

df_clean = df.dropna(subset=['Price', 'Review Scores Rating']).copy()
y_cv = df_clean['Price']
X3_cv = df_clean[['Review Scores Rating']]
X4_cv = pd.get_dummies(df_clean, columns=['Neighbourhood'], drop_first=

features4 = ['Review Scores Rating'] + [
    col for col in X4_cv.columns if col.startswith('Neighbourhood_')
]

```

```

X4_cv = X4_cv[features4].astype(float)

df_dummies_all_cv = pd.get_dummies(df_clean, columns=['Neighbourhood'])

X5_cv = pd.DataFrame()

for col in [c for c in df_dummies_all_cv.columns if c.startswith('Neig
X5_cv[f"Slope_{col.replace('Neighbourhood_', '')}"] = (
    df_dummies_all_cv[col] * df_dummies_all_cv['Review Scores Rati
    )

X5_cv = X5_cv.astype(float)

kf = KFold(n_splits=5, shuffle=True, random_state=42)
lr = LinearRegression()

def get_rmse(X, y):
    neg_mse = cross_val_score(lr, X, y, cv=kf, scoring='neg_mean_squar
    return np.mean(np.sqrt(-neg_mse))

print("==== Cross Validation Results (RMSE) ====")
print("Model 3 (Review Scores Only): $", round(get_rmse(X3_cv, y_cv),
print("Model 4 (Scores + Neighbourhood): $", round(get_rmse(X4_cv, y_c
print("Model 5 (Scores * Neighbourhood Puzzle): $", round(get_rmse(X5_

```

```

==== Cross Validation Results (RMSE) ====
Model 3 (Review Scores Only): $ 144.17
Model 4 (Scores + Neighbourhood): $ 140.52
Model 5 (Scores * Neighbourhood Puzzle): $ 140.43

```

Q3. This question is a case study for linear models. The data are about car prices.

In particular, they include:

- **Price** : In Indian rupees
- **Seating_Capacity** : Number of seats
- **Body_Type** : crossover, hatchback, muv, sedan, suv
- **Make_Year** : The year the car was made

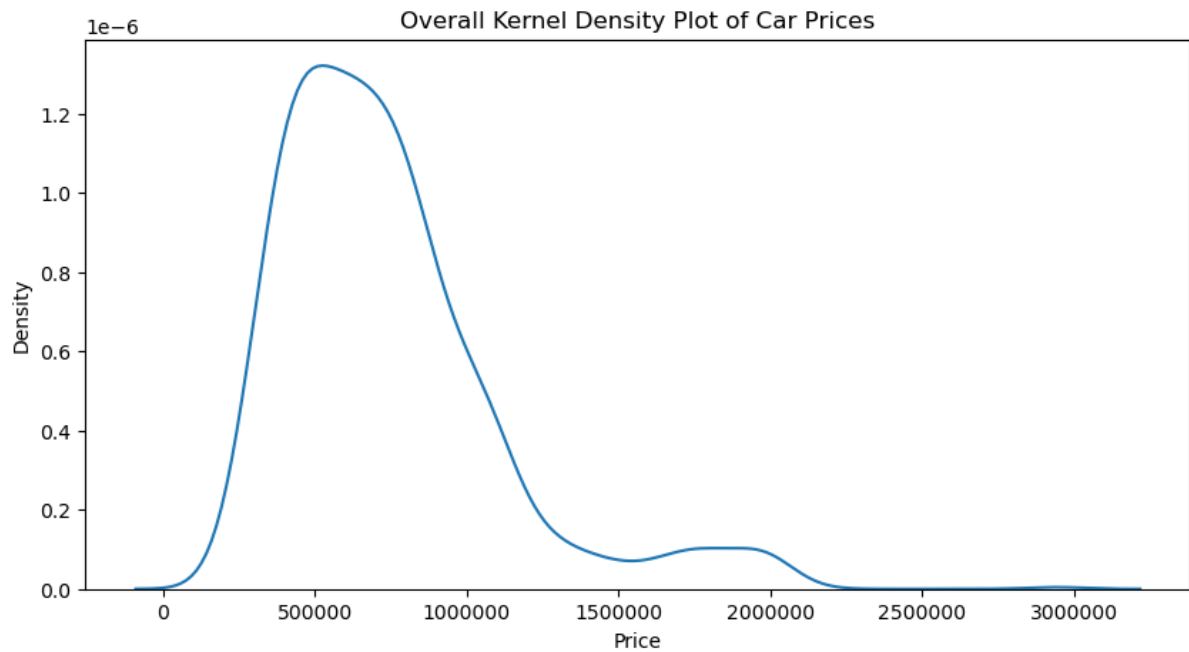
1. Load **cars_hw.csv** . Summarize the **Price** variable and create a kernel density plot. Use **.groupby()** and **.describe()** to summarize prices by **Body_Type** . Make a grouped kernel density plot by **Body_Type** . Which car types are the most expensive? Which have the most variance?
2. Regress **Price** on **Seating_Capacity** . What's the slope coefficient? Interpret it. Now treat **Seating_Capacity** as a one hot encoded variable, and regress price on it as if it is categorical. Are the differences in price roughly linear in the number of seats?
3. Use **Make_Year** to create a new variable that corresponds to the age of the vehicle. Use 10-fold cross validation to determine the optimal number of powers of **Age** to include in a regression of **Price** on **Age** .

4. Plot **Price** against **Age** , and then model-predicted price against **Age** .

Does the model accurately fit the patterns in the data?

```
In [12]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
cars = pd.read_csv('./data/cars_hw.csv')
print("=== Overall Price Summary ===")
print(cars['Price'].describe())
print("\n")
plt.figure(figsize=(10, 5))
sns.kdeplot(data=cars, x='Price')
plt.title('Overall Kernel Density Plot of Car Prices')
plt.ticklabel_format(style='plain', axis='x')
plt.show()
print("=== Price Summary by Body_Type ===")
summary_by_body = cars.groupby('Body_Type')['Price'].describe()
print(summary_by_body)
plt.figure(figsize=(10, 5))
sns.kdeplot(data=cars, x='Price', hue='Body_Type')
plt.title('Kernel Density Plot of Car Prices Grouped by Body Type')
plt.ticklabel_format(style='plain', axis='x')
plt.show()
```

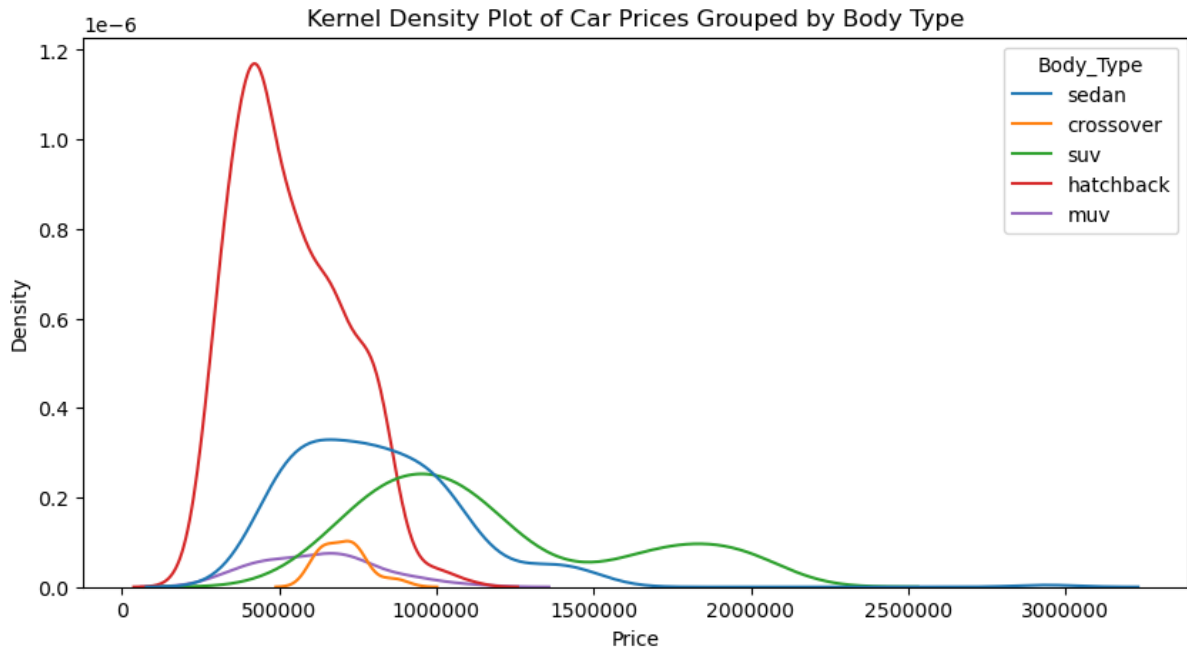
```
=== Overall Price Summary ===
count    9.760000e+02
mean     7.410195e+05
std      3.673234e+05
min      1.880000e+05
25%      4.757500e+05
50%      6.665000e+05
75%      8.830000e+05
max      2.941000e+06
Name: Price, dtype: float64
```



=== Price Summary by Body_Type ===

	count	mean	std	min	25%	
50% \						
Body_Type						
crossover	21.0	7.050952e+05	72849.780207	608000.0	638000.0	706
000.0						
hatchback	484.0	5.339773e+05	173086.729837	188000.0	401750.0	509
000.0						
muv	38.0	6.264211e+05	184093.488526	328000.0	489750.0	621
500.0						
sedan	227.0	8.097841e+05	286736.770092	368000.0	598000.0	772
000.0						
suv	206.0	1.176495e+06	417479.028217	559000.0	860000.0	1065
000.0						

	75%	max
Body_Type		
crossover	749000.0	883000.0
hatchback	661000.0	1107000.0
muv	738500.0	1091000.0
sedan	968500.0	2941000.0
suv	1491250.0	2100000.0



```
In [13]: #2
import pandas as pd
import statsmodels.api as sm
print("=== Regression summary treating seats as a continuous variable")
X_cont = sm.add_constant(cars['Seating_Capacity'])
model_cont = sm.OLS(cars['Price'], X_cont, missing='drop').fit()
print(model_cont.summary())
print("\n")
print("=== Regression summary treating seats as a categorical variable")
cars_dummies = pd.get_dummies(cars, columns=['Seating_Capacity'], drop
features_cat = [col for col in cars_dummies.columns if col.startswith(
X_cat = sm.add_constant(cars_dummies[features_cat].astype(float))
model_cat = sm.OLS(cars['Price'], X_cat, missing='drop').fit()
print(model_cat.summary())
```

```
=== Regression summary treating seats as a continuous variable ===
                        OLS Regression Results
```

```
=====
=====
Dep. Variable:                Price    R-squared:
0.005
Model:                        OLS      Adj. R-squared:
0.004
Method:                       Least Squares    F-statistic:
5.075
Date:                          Mon, 23 Mar 2026    Prob (F-statistic):
0.0245
Time:                          00:07:11    Log-Likelihood:
-13888.
No. Observations:                976    AIC:                2.
778e+04
Df Residuals:                    974    BIC:                2.
779e+04
```

```

Df Model:                                1
Covariance Type:                        nonrobust
=====
=====
              coef      std err          t      P>|t|      [0.02
5      0.975]
-----
const                4.39e+05    1.35e+05     3.263     0.001    1.75e+0
5      7.03e+05
Seating_Capacity    5.927e+04    2.63e+04     2.253     0.024    7641.18
9      1.11e+05
=====
=====
Omnibus:                        314.988    Durbin-Watson:
1.430
Prob(Omnibus):                  0.000    Jarque-Bera (JB):
919.828
Skew:                           1.627    Prob(JB):                        1.
83e-200
Kurtosis:                       6.468    Cond. No.
60.9
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

=== Regression summary treating seats as a categorical variable ===
 OLS Regression Results

```

=====
=====
Dep. Variable:                    Price    R-squared:
0.022
Model:                            OLS    Adj. R-squared:
0.018
Method:                        Least Squares    F-statistic:
5.397
Date:                Mon, 23 Mar 2026    Prob (F-statistic):
0.000267
Time:                00:07:11    Log-Likelihood:
-13880.
No. Observations:                976    AIC:                        2.
777e+04
Df Residuals:                    971    BIC:                        2.
779e+04
Df Model:                        4
Covariance Type:                nonrobust
=====
=====

```

	coef	std err	t	P> t	[0.
025 0.975]					

const	1.88e+05	3.64e+05	0.516	0.606	-5.26e
+05 9.02e+05					
Seating_Capacity_5	5.465e+05	3.64e+05	1.500	0.134	-1.68e
+05 1.26e+06					
Seating_Capacity_6	1.456e+06	5.15e+05	2.828	0.005	4.46e
+05 2.47e+06					
Seating_Capacity_7	7.359e+05	3.69e+05	1.996	0.046	1.24e
+04 1.46e+06					
Seating_Capacity_8	2.718e+05	3.99e+05	0.682	0.496	-5.11e
+05 1.05e+06					
=====					
Omnibus:	324.467	Durbin-Watson:			
1.394					
Prob(Omnibus):	0.000	Jarque-Bera (JB):			
981.918					
Skew:	1.663	Prob(JB):			6.
01e-214					
Kurtosis:	6.618	Cond. No.			
99.7					
=====					
=====					

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [14]: #3
import pandas as pd
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import LinearRegression
cars = cars.dropna(subset=['Price', 'Make_Year']).copy()
cars['Age'] = 2026 - cars['Make_Year']
X = cars[['Age']]
y = cars['Price']
kf = KFold(n_splits=10, shuffle=True, random_state=42)
lr = LinearRegression()
print("=== 10-Fold CV RMSE by Polynomial Degree of Age ===")
best_degree = 1
min_rmse = float('inf')
for degree in range(1, 9):
    poly = PolynomialFeatures(degree=degree)
    X_poly = poly.fit_transform(X)

    neg_mse = cross_val_score(lr, X_poly, y, cv=kf, scoring='neg_mean_
    rmse = np.mean(np.sqrt(-neg_mse))
    print(f"Degree {degree}: ${round(rmse, 2)}")
```

```

if rmse < min_rmse:
    min_rmse = rmse
    best_degree = degree
print(f"\n=> The optimal number of powers (lowest RMSE) is: {best_degr

```

=== 10-Fold CV RMSE by Polynomial Degree of Age ===

```

Degree 1: $310765.36
Degree 2: $310779.3
Degree 3: $309697.34
Degree 4: $308871.57
Degree 5: $307368.98
Degree 6: $307522.52
Degree 7: $306433.42
Degree 8: $306919.53

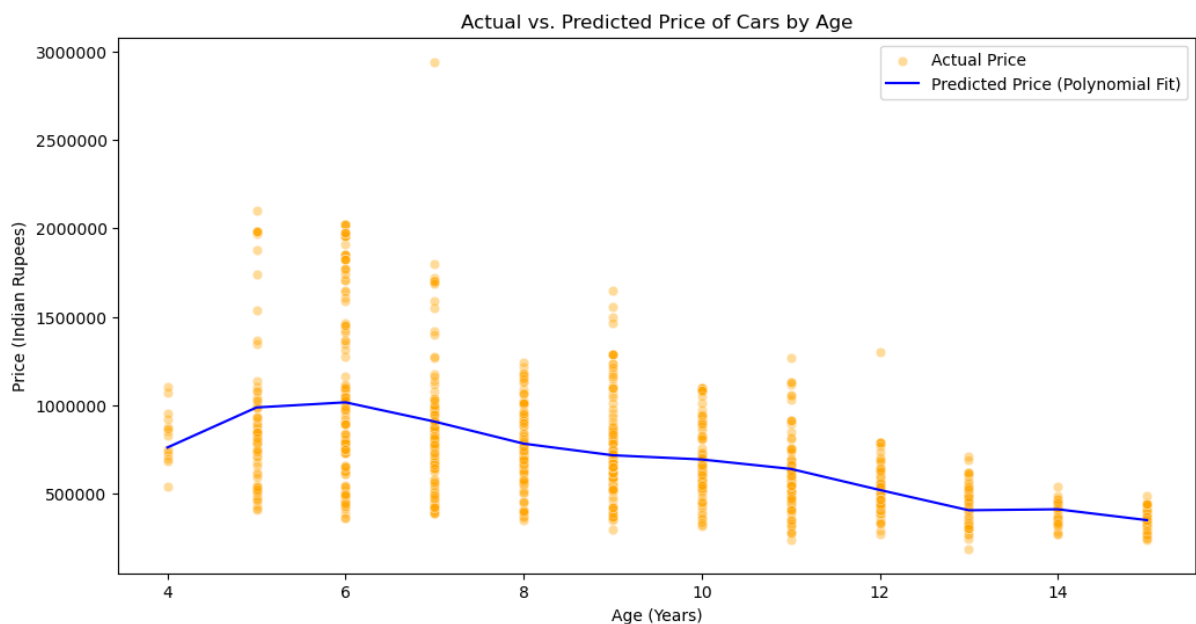
```

=> The optimal number of powers (lowest RMSE) is: 8

```

In [17]: #4
poly = PolynomialFeatures(degree=best_degree)
X_poly = poly.fit_transform(X)
final_model = lr.fit(X_poly, y)
cars['Predicted_Price'] = final_model.predict(X_poly)
plt.figure(figsize=(12, 6))
sns.scatterplot(data=cars, x='Age', y='Price', color='orange', alpha=0.5)
cars_sorted = cars.sort_values(by='Age')
plt.plot(cars_sorted['Age'], cars_sorted['Predicted_Price'], color='blue')
plt.title('Actual vs. Predicted Price of Cars by Age')
plt.xlabel('Age (Years)')
plt.ylabel('Price (Indian Rupees)')
plt.ticklabel_format(style='plain', axis='y')
plt.legend()
plt.show()

```



4

Yes. A polynomial regression captures the non-linear depreciation of cars (losing value rapidly when new, then the curve flattening out as they age), whereas a standard straight line would completely miss this trend. There may be slight upward at extreme older ages, which is a normal artifact of high-degree polynomials in areas with sparse data.

Q4. This question refers to the `heart_hw.csv` data. It contains three variables:

- `y` : Whether the individual survived for three years, coded 0 for death and 1 for survival
- `age` : Patient's age
- `transplant` : labelled `control` for not receiving a transplant and labelled `treatment` for receiving a transplant

Since a heart transplant is a dangerous operation and even people who successfully get heart transplants might suffer later complications, we want to look at whether a group of transplant recipients tends to survive longer than a comparison group who does not get the procedure.

1. Compute (a) the proportion of people who survive in the control group and (b) in the treatment group. Compute (b) minus (a) as the average treatment effect (ATE). What is the ATE on three-year survival for heart transplant interventions?
2. Regress `y` on `transplant` using a linear model with a constant. Compare the intercept and transplant coefficient to the numbers you computed in part 1.
3. Regress `y` on `transplant` and `age`. How does the coefficient on `transplant` change when you control for `age`? What do the intercept and slope represent? Plot the predicted probability of survival by age, and add a dashed horizontal line for the ATE from part 1. For what ages is the ATE over- or under-estimating the impact of a transplant?
4. Now, include `transplant`, `age`, and `transplant * age` as controls. Repeat your analysis from part 3. How does allowing `age` and `transplant` to interact change your predictions? What pattern do you notice?
5. Use 10-fold cross validation to evaluate the predictive accuracy of these models.
6. Imagine this model is used to prioritize transplant access. What are your concerns about model construction and deployment?

```
In [19]: #1
import pandas as pd
heart = pd.read_csv('./data/heart_hw.csv')
survival_rates = heart.groupby('transplant')['y'].mean()
control_rate = survival_rates['control']
treatment_rate = survival_rates['treatment']
ate = treatment_rate - control_rate
print(f"(A) Control survival rate: {control_rate:.4f} ({control_rate*100:.2f}%)")
print(f"(B) Treatment survival rate: {treatment_rate:.4f} ({treatment_rate*100:.2f}%)")
print(f"Average Treatment Effect (ATE): {ate:.4f} ({ate*100:.2f}%)")
```

(A) Control survival rate: 0.1176 (11.76%)

(B) Treatment survival rate: 0.3478 (34.78%)

Average Treatment Effect (ATE): 0.2302 (23.02%)

The Average Treatment Effect (ATE) is 0.2302 or 23.02%. This means that receiving a heart transplant intervention increases the probability of surviving for three years by an absolute margin of 23.02% over the control group.

```
In [20]: #2
import statsmodels.api as sm
heart_dummies = pd.get_dummies(heart, columns=['transplant'], drop_first=True)
X = heart_dummies[['transplant_treatment']].astype(float)
X = sm.add_constant(X)
y = heart_dummies['y']
model_heart = sm.OLS(y, X).fit()
print(model_heart.summary())
```

OLS Regression Results

```

=====
Dep. Variable:          y      R-squared:
0.059
Model:                  OLS    Adj. R-squared:
0.050
Method:                 Least Squares    F-statistic:
6.354
Date:                   Mon, 23 Mar 2026    Prob (F-statistic):
0.0133
Time:                   00:12:16    Log-Likelihood:
-59.591
No. Observations:      103    AIC:
123.2
Df Residuals:          101    BIC:
128.5
Df Model:               1
Covariance Type:       nonrobust
=====
=====

```

	coef	std err	t	P> t	[
const	0.1176	0.075	1.574	0.119	-
transplant_treatment	0.2302	0.091	2.521	0.013	

```

=====
Omnibus:                22.540    Durbin-Watson:
1.416
Prob(Omnibus):          0.000    Jarque-Bera (JB):
17.584
Skew:                   0.902    Prob(JB):
0.000152
Kurtosis:               2.080    Cond. No.
3.24
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

In [22]: #3
import statsmodels.api as sm
import matplotlib.pyplot as plt
import numpy as np
# 1. Prepare Features (Age and Transplant dummy variable)
heart_dummies = pd.get_dummies(heart, columns=['transplant'], drop_first=True)
X3 = heart_dummies[['transplant_treatment', 'age']].astype(float)

```

```
X3 = sm.add_constant(X3)
y = heart_dummies['y']
# 2. Fit Regression Model
model3 = sm.OLS(y, X3).fit()
print(model3.summary())
# 3. Create data for the plot
# We create a range of ages from the minimum age to maximum age in the
age_range = np.linspace(heart['age'].min(), heart['age'].max(), 100)
# Calculate predicted lines using the coefficients from model3
intercept = model3.params['const']
transplant_coef = model3.params['transplant_treatment']
age_coef = model3.params['age']
# Control line equation: Intercept + Age_Coef * age
prob_control = intercept + (age_coef * age_range)
# Treatment line equation: Intercept + Transplant_Coef + Age_Coef * age
prob_treatment = intercept + transplant_coef + (age_coef * age_range)
# 4. Draw the Plot
plt.figure(figsize=(10, 6))
plt.plot(age_range, prob_control, color='red', label='Predicted Survival Control')
plt.plot(age_range, prob_treatment, color='blue', label='Predicted Survival Treatment')
# We plot the ATE (0.2302) as a horizontal dashed line as requested
plt.axhline(y=0.2302, color='green', linestyle='--', label='ATE from P')
plt.title('Predicted Probability of Survival by Age')
plt.xlabel('Age')
plt.ylabel('Probability of Survival')
plt.legend()
plt.show()
```

OLS Regression Results

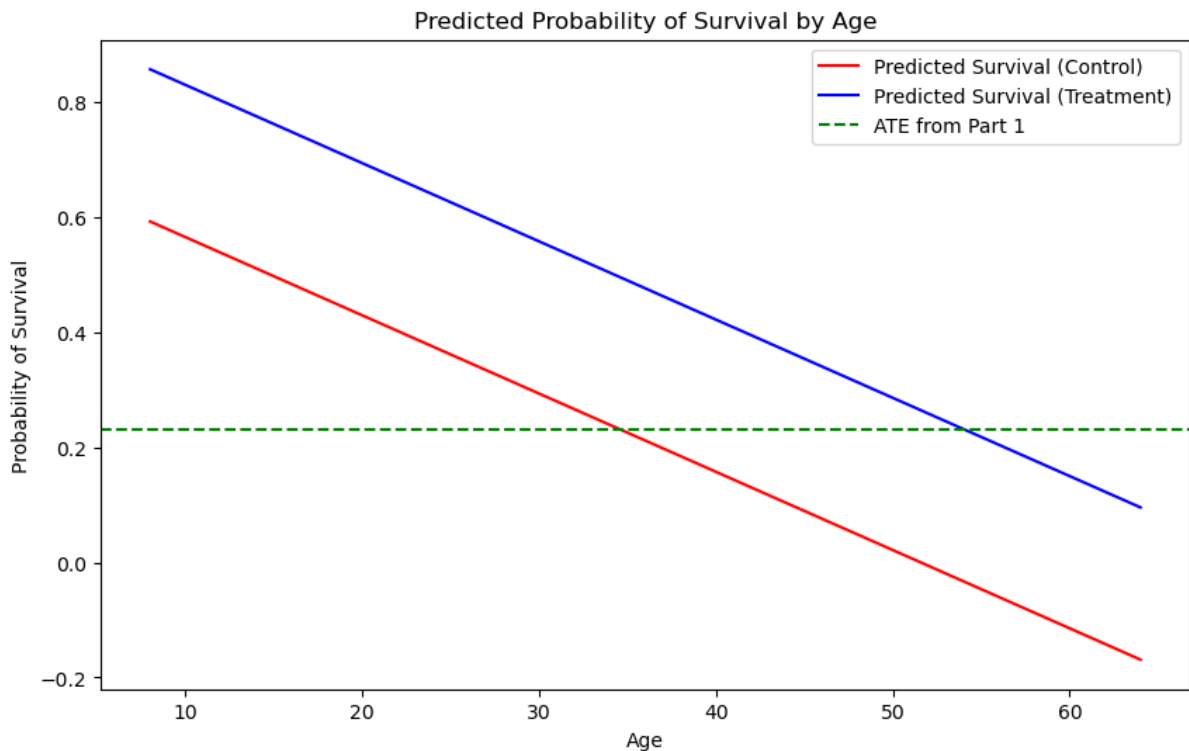
```

=====
=====
Dep. Variable:          y      R-squared:
0.147
Model:                  OLS    Adj. R-squared:
0.130
Method:                 Least Squares    F-statistic:
8.602
Date:                   Mon, 23 Mar 2026    Prob (F-statistic):
0.000357
Time:                   00:13:25    Log-Likelihood:
-54.558
No. Observations:      103    AIC:
115.1
Df Residuals:          100    BIC:
123.0
Df Model:               2
Covariance Type:       nonrobust
=====
=====
                                coef    std err          t      P>|t|      [
0.025    0.975]
-----
const                0.7020    0.196      3.583    0.001
0.313    1.091
transplant_treatment  0.2647    0.088      3.006    0.003
0.090    0.439
age                 -0.0136    0.004     -3.204    0.002    -
0.022   -0.005
=====
=====
Omnibus:              14.464    Durbin-Watson:
1.370
Prob(Omnibus):        0.001    Jarque-Bera (JB):
12.288
Skew:                 0.753    Prob(JB):
0.00215
Kurtosis:             2.227    Cond. No.
219.
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.



3

Because this specific model does not include an interaction between age and transplant, the new model assumes the impact of the transplant is exactly 26.47% for all ages. Therefore, the original naive ATE (23.02%) is actively under-estimating the impact of the transplant for every single age group.

```
In [23]: #4
import statsmodels.api as sm
import matplotlib.pyplot as plt
import numpy as np
heart_dummies = pd.get_dummies(heart, columns=['transplant'], drop_first=True)
X4 = heart_dummies[['transplant_treatment', 'age']].astype(float)
X4['transplant_x_age'] = X4['transplant_treatment'] * X4['age']
X4 = sm.add_constant(X4)
y = heart_dummies['y']
model4 = sm.OLS(y, X4).fit()
print(model4.summary())
age_range = np.linspace(heart['age'].min(), heart['age'].max(), 100)
intercept = model4.params['const']
transplant_coef = model4.params['transplant_treatment']
age_coef = model4.params['age']
interaction_coef = model4.params['transplant_x_age']
prob_control_int = intercept + (age_coef * age_range)
prob_treatment_int = intercept + transplant_coef + (age_coef * age_range)
plt.figure(figsize=(10, 6))
plt.plot(age_range, prob_control_int, color='red', label='Predicted Su
```

```
plt.plot(age_range, prob_treatment_int, color='blue', label='Predicted')
plt.title('Predicted Probability of Survival (Interactive Model)')
plt.xlabel('Age')
plt.ylabel('Probability of Survival')
plt.legend()
plt.show()
```

OLS Regression Results

```

=====
Dep. Variable:          y      R-squared:
0.172
Model:                  OLS    Adj. R-squared:
0.147
Method:                 Least Squares    F-statistic:
6.839
Date:                   Mon, 23 Mar 2026    Prob (F-statistic):
0.000310
Time:                   00:14:25    Log-Likelihood:
-53.033
No. Observations:      103    AIC:
114.1
Df Residuals:          99    BIC:
124.6
Df Model:               3
Covariance Type:       nonrobust
=====

```

```

=====
coef      std err      t      P>|t|      [
0.025      0.975]
-----
const              0.3549      0.280      1.269      0.207      -
0.200      0.910
transplant_treatment      0.9078      0.383      2.370      0.020
0.148      1.668
age              -0.0055      0.006     -0.877      0.382      -
0.018      0.007
transplant_x_age      -0.0146      0.008     -1.725      0.088      -
0.031      0.002
=====

```

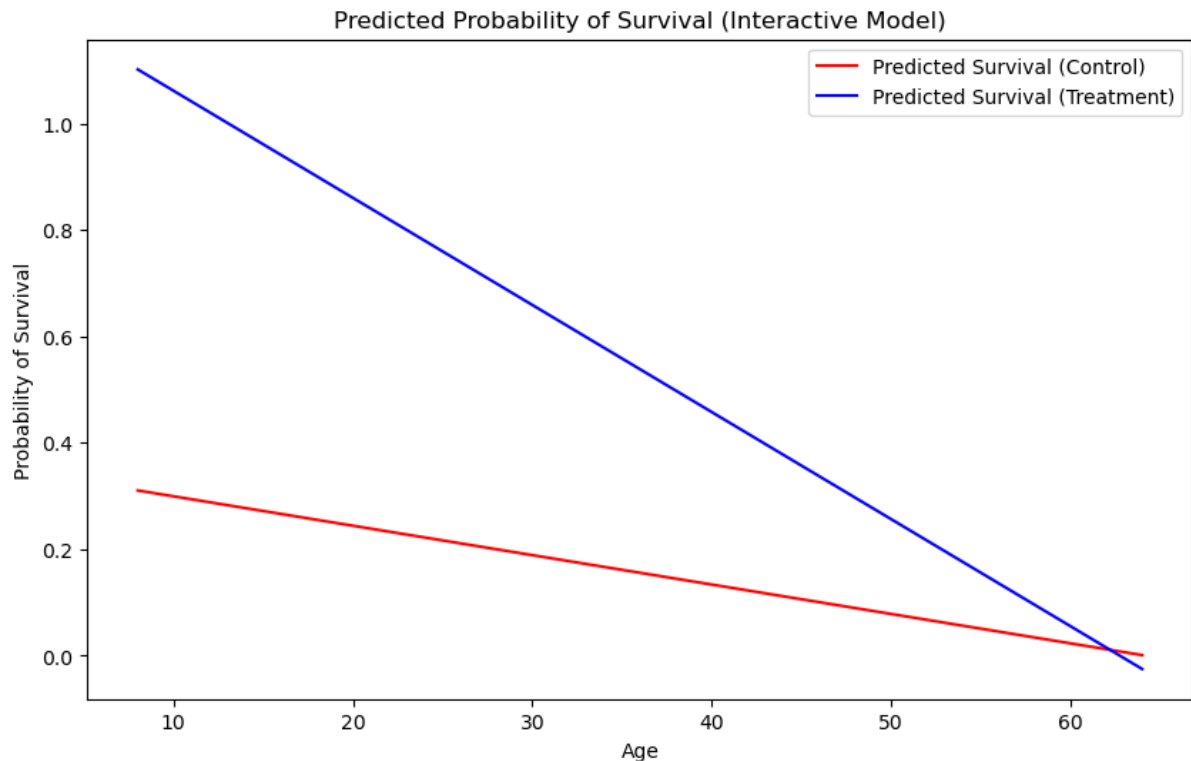
```

=====
Omnibus:              10.652    Durbin-Watson:
1.312
Prob(Omnibus):        0.005    Jarque-Bera (JB):
11.705
Skew:                 0.803    Prob(JB):
0.00287
Kurtosis:             2.615    Cond. No.
622.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.



4

It prevents the model from assuming the transplant helps everyone equally. By adding the interaction term, the survival prediction lines are no longer perfectly parallel—they now have different slopes. The interaction term (`transplant_x_age`) has a strongly negative coefficient. This pattern reveals that the transplant is massively beneficial for younger patients, but that survival benefit rapidly deteriorates as the patient's age increases. Eventually, the two lines actually intersect (around age 62), indicating that for elderly patients, undergoing the major surgery of a transplant may actually be more deadly than the control approach.

```
In [24]: #5
import pandas as pd
from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import LinearRegression
import numpy as np
heart = pd.read_csv('./data/heart_hw.csv')
heart_clean = heart.dropna(subset=['y', 'transplant', 'age']).copy()
heart_dummies_cv = pd.get_dummies(heart_clean, columns=['transplant'],
y_cv = heart_dummies_cv['y']
X2_cv = heart_dummies_cv[['transplant_treatment']]
X3_cv = heart_dummies_cv[['transplant_treatment', 'age']]
X4_cv = X3_cv.copy()
X4_cv['transplant_x_age'] = X4_cv['transplant_treatment'] * X4_cv['age']
```

```

kf = KFold(n_splits=10, shuffle=True, random_state=42)
lr = LinearRegression()
def get_rmse(X, y):
    neg_mse = cross_val_score(lr, X, y, cv=kf, scoring='neg_mean_squared_error')
    return np.mean(np.sqrt(-neg_mse))
print("==== 10-Fold Cross Validation Results (RMSE) ====")
print(f"Model 2 (Transplant Only): {get_rmse(X2_cv, y_cv):.4f}")
print(f"Model 3 (Transplant + Age): {get_rmse(X3_cv, y_cv):.4f}")
print(f"Model 4 (Interactive): {get_rmse(X4_cv, y_cv):.4f}")

```

```

==== 10-Fold Cross Validation Results (RMSE) ====
Model 2 (Transplant Only): 0.4365
Model 3 (Transplant + Age): 0.4167
Model 4 (Interactive): 0.4130

```

6

This model has two major flaws. First, it relies solely on age, ignoring critical medical factors like illness severity, compatibility, immune health, and wait times — which structurally discriminates against older patients regardless of their individual circumstances. Second, since the outcome is binary, linear regression is the wrong tool; its unbounded output can produce impossible probabilities like -15%, making it dangerously unreliable for medical decisions. Logistic regression should be used instead.